

DAY 12

TECHNICAL REFERENCE

REST APIs & Resource Reference

A plain-language guide to REST, and the complete resource and endpoint map for the AI Resume Builder.

Technology Stack

Angular • Express.js (Node) • MySQL



Dinesh Rawat

linkedin.com/in/dinesh-rawat

Companion to the AI Resume Builder PRD v1.0

Category: Career Tools / SaaS

1. What a REST API Is (explained simply)

Imagine a big restaurant. You sit at a table, and you never walk into the kitchen. A waiter carries your order to the kitchen and brings the food back. You and the kitchen never talk directly. The waiter does all the running back and forth.

A REST API is that waiter. In this app, **you** are the screen you see (the Angular part), the **kitchen** is the powerful computer that stores everything (the Express and MySQL part), and the **API** is the waiter that carries messages between them.

Every "thing" has an address

Every item the app cares about, one resume, one template, one saved job, is a "thing." And every thing has an address, like a house on a street. The address `/api/documents/42` just means "the resume numbered 42." Like a house number, it points to exactly one thing.

You only do four jobs to a thing

No matter what, there are really only four things you can ask the waiter to do. That is all.

Word	What it means	Like saying...
GET	Show me the thing	"Can I see my resume?"
POST	Make a new thing	"Please make me a new resume."
PUT	Change the thing	"Fix this part of my resume."
DELETE	Throw the thing away	"Delete this resume."

Here is the clever part: the address stays the same, but the *word* changes what happens. Say GET to house 42 and you see the resume. Say DELETE to the same house 42 and you throw it away. Same address, different job, just a different word.

The waiter forgets you each time

Every time you call the waiter, you have to say who you are again, because the waiter does not remember your last order. That sounds annoying, but it is actually good: the restaurant can hire lots of waiters, and any of them can help you. That is how the app serves thousands of people at once.

They pass notes in a tidy way

The messages the waiter carries are written in a neat format called JSON, which is just a tidy way of writing down facts so both sides understand it, like a form with the blanks filled in.

And the kitchen always sends back a little number that says how it went: **200** means "here you go," **201** means "made it," **404** means "I could not find that thing," and **500** means "oops, the kitchen dropped something."

Two quick words: a **resource** is a "thing" (a resume, a template). An **endpoint** is one address plus one job word, like `POST /api/documents` ("make a new resume"). The rest of this guide lists every thing, and every job you can do to it.

2. Resources in the Resume Builder

Working from the PRD data model, these are the nouns the system manages. Some are stored data; a few (auth, AI, ATS, tailoring, export) are actions that produce a result rather than a saved row.

#	Resource	What it represents
1	Auth / session	Registration, login, tokens. The entry point, not stored data.
2	User	The account, its tier, and AI usage.
3	Document	A resume or cover letter. The central resource.
4	Section	A block within a document (experience, education, skills).
5	Section item	One entry inside a section (a single job, a single degree).
6	Version	A saved or tailored snapshot of a document.
7	Template	An available design.
8	AI	Writing actions (bullets, summary, rewrite). An action, not stored data.
9	ATS check	A scoring action.
10	Tailoring	Align a resume to a job description.
11	Export	A generated PDF or DOCX file.
12	Share	A public link to a document.
13	Application	A tracked job application.

3. Full API Surface

Endpoints grouped by resource. This expands the summary table from the PRD into the complete set.

Auth

Endpoint	Purpose
POST /api/auth/register	Create account
POST /api/auth/login	Obtain an access token
POST /api/auth/logout	Invalidate the session
POST /api/auth/forgot-password	Begin password recovery
POST /api/auth/reset-password	Complete password reset

Users

Endpoint	Purpose
GET /api/users/me	Current profile, tier, AI credits
PUT /api/users/me	Update profile
DELETE /api/users/me	Delete account and data

Documents (the core resource)

Endpoint	Purpose
GET /api/documents	List my resumes and cover letters
POST /api/documents	Create one (blank or from a template)
POST /api/documents/import	Create one by parsing an upload or LinkedIn data
GET /api/documents/:id	Read one with its full content
PUT /api/documents/:id	Save edits
POST /api/documents/:id/duplicate	Copy it (to spin off a tailored version)
DELETE /api/documents/:id	Delete it

Sections & items (nested under a document)

Endpoint	Purpose
POST /api/documents/:id/sections	Add a section
PATCH /api/documents/:id/sections/:sectionId	Edit or reorder a section
DELETE /api/documents/:id/sections/:sectionId	Remove a section
POST /api/.../sections/:sectionId/items	Add an entry
PATCH /api/.../items/:itemId	Edit or reorder an entry
DELETE /api/.../items/:itemId	Remove an entry

In practice many builders save the whole document in one **PUT** `/api/documents/:id` call rather than editing each section over the wire. Both are valid. The nested routes matter more if you want fine-grained, autosave-per-field behavior.

Versions

Endpoint	Purpose
GET /api/documents/:id/versions	List saved versions
POST /api/documents/:id/versions	Save the current state as a version
POST /api/documents/:id/versions/:versionId/restore	Roll back to one

Templates

Endpoint	Purpose
GET /api/templates	List available designs
GET /api/templates/:id	One template's config

AI (action resource; each call is metered)

Endpoint	Purpose
POST /api/ai/bullets	Generate or improve bullet points
POST /api/ai/summary	Generate a summary or headline
POST /api/ai/rewrite	Tighten or improve selected text
POST /api/ai/prompt	Apply a freeform instruction to a section

ATS check, Tailoring, Exports

Endpoint	Purpose
POST /api/ats/check	Score a document, return issues (basic free, full for Pro)
POST /api/tailoring	Take a document + job description, return suggested edits (Pro)
POST /api/exports/pdf	Render a PDF, return its file URL
POST /api/exports/docx	Render editable Word
GET /api/exports/:id	Retrieve a past export

Sharing

Endpoint	Purpose
POST /api/documents/:id/share	Create or refresh a public link
DELETE /api/documents/:id/share	Revoke it
GET /api/share/:slug	Public, unauthenticated view of a shared resume

Applications (tracker)

Endpoint	Purpose
GET /api/applications	List tracked applications
POST /api/applications	Log one
PATCH /api/applications/:id	Update status
DELETE /api/applications/:id	Remove one

4. Patterns Worth Noticing

- **Nesting shows ownership.** Items live under sections, which live under documents, so the URL reads like a path to the thing.
- **Non-CRUD work is modeled as POST to a verb-like resource.** AI, ATS, tailoring, and export create a result even though the "thing" is a computation rather than a saved row.
- **One endpoint is deliberately public.** `GET /api/share/:slug` skips auth on purpose, because that is the point of a shareable link. It uses an unguessable slug instead of the raw document ID so records cannot be enumerated.
- **Verbs carry the intent, URLs carry the identity.** The same URL supports read, update, and delete depending only on the verb.

5. Bonus: Where a Browser Stores Data

Your resume builder runs in a browser, and browsers can save little bits of data on the user's own computer. If you press **F12 → Application** in Chrome, you will see several places it can store things. Here is what each one is for, explained simply.

Think of the browser like a school bag. It has different pockets, and each pocket holds a different kind of thing.

```
Browser
|
|-- Cookies
|-- Local Storage
|-- Session Storage
|-- IndexedDB
```

1. Cookies

A cookie is like a visitor badge at a mall. The guard gives you badge "Visitor #102", and every shop you enter sees it and knows you are the same person.

When you log into Gmail, the browser saves a small cookie like `user = dinesh`, `token = abc123`. Next time you visit, the browser automatically sends that cookie, and the server says "yes, I know you," so you do not log in again.

Key traits: very small (about 4 KB), **sent to the server automatically** on every request, and can expire (for example after 7 days).

Used for: login, authentication, "remember me," shopping carts, language choice.

2. Local Storage

Local Storage is like a drawer in your room. Whatever you put in stays there until you take it out, tomorrow, next week, next month, it is still there.

You pick Dark Mode, the site saves `theme = dark`. Come back any day later, it reads `theme = dark` and dark mode is back.

Key traits: around 5–10 MB, never expires, **not sent to the server**, only your browser reads it.

```
// save
localStorage.setItem("theme", "dark");
// read
localStorage.getItem("theme");
// remove
localStorage.removeItem("theme");
```

3. Session Storage

Session Storage is like writing on a classroom whiteboard. When class ends, the teacher wipes it clean. Gone.

While you fill an online form, the site can hold your input temporarily. Refresh by accident and it is still there. But close the tab and everything disappears. Every tab even gets its own separate copy: search "Laptop" in one Amazon tab and "Shoes" in another, and they do not share.

Key traits: around 5 MB, lives only while the tab is open, **not sent to the server**, separate per tab.

```
sessionStorage.setItem("name", "Dinesh");
sessionStorage.getItem("name");
```

4. IndexedDB

IndexedDB is like a whole library, thousands of books, millions of pages, all organized. If Local Storage is one drawer, IndexedDB is the entire library.

It holds big things. Google Docs keeps your document here so it survives when the internet drops. Spotify stores downloaded songs here. Google Maps keeps offline maps here, because they are far too big for cookies or Local Storage.

Key traits: hundreds of MB or more, and unlike the others it can store whole objects, images, files, and videos directly, not just text.

Quick Comparison

Feature	Cookies	Local Storage	Session Storage	IndexedDB
Size	4 KB	5–10 MB	5 MB	Hundreds of MB+
Sent to server	Yes	No	No	No
Expires	Yes	No	On tab close	No
Large data	No	No	No	Yes
Stores objects	No	Strings only	Strings only	Yes
Best use	Login, auth	Theme, prefs	Temp form data	Offline, large data

Easy Memory Trick

- **Cookie = Visitor Badge** — small, sent to server, used for login.
- **Local Storage = Bedroom Drawer** — stays forever, good for settings.
- **Session Storage = Classroom Whiteboard** — gone when the tab closes, good for temporary data.
- **IndexedDB = Library** — huge, good for offline apps, files, images, and big datasets.

How Real Sites Use Them

Website	Storage used	Why
Gmail	Cookies	Keep you logged in
YouTube	Local Storage	Remember dark mode, volume, preferences
Amazon	Cookies + Local Storage	Login, cart, preferences
Google Docs	IndexedDB	Save documents offline
Spotify	IndexedDB	Cache downloaded songs
Google Maps	IndexedDB	Offline maps

Interview Questions

- Which storage is sent to the server on every request? **Cookies.**
- Which survive a browser restart? **Local Storage and IndexedDB.**
- Which is deleted when the tab closes? **Session Storage.**
- Which is best for offline apps and large data? **IndexedDB.**
- Can Local Storage store objects directly? **No** — strings only. Use `JSON.stringify()` to save and `JSON.parse()` to read back. IndexedDB stores objects directly.